Abstract

This project implements a distributed big data pipeline for detecting AI-generated Arabic abstracts using Apache Spark. The system processes the "Arabic Generated Abstracts" dataset from Hugging Face, applying comprehensive Arabic text preprocessing including diacritic removal, character normalization, stopword filtering, and optional camel-tools integration for advanced morphological analysis. Four classification models Logistic Regression, Naive Bayes, Random Forest, and Linear SVM are trained and evaluated. Logistic Regression achieved the highest validation F1-score (0.9804) with an AUC of 0.9955. The best model achieved 98.25% accuracy on the held-out test set. A file-based streaming simulation demonstrates real-time inference capability, and scalability analysis shows optimal performance at 8 partitions. The project confirms that combining TF-IDF with syllable-level and morphological features is highly effective for Arabic AI-text detection.

1.Introduction

The rapid advancement of Large Language Models (LLMs) capable of generating fluent, contextually relevant Arabic text presents significant challenges for content authentication and information integrity. Academic institutions, publishers, and content platforms increasingly face the threat of AI-generated submissions that bypass traditional detection methods. Unlike English, where substantial research has been devoted to machine-generated text detection, Arabic a morphologically rich language with complex writing systems and significant dialectal variation remains underserved by existing detection frameworks. This disparity creates a critical gap, as malicious actors could exploit the relative scarcity of Arabic detection tools to produce synthetic content for academic fraud, misinformation campaigns, or automated social media manipulation. The growing availability of Arabic-capable LLMs, including models such as Allam, Jais, Llama, and OpenAI, further amplifies this risk, demanding the development of robust, scalable detection systems tailored to Arabic linguistic characteristics.

This project objectives:

1. To design and implement a distributed data processing pipeline for Arabic text analysis using Apache Spark
2. To develop comprehensive Arabic text preprocessing functions—including diacritic removal, character normalization, stopword filtering, and stemming—implemented as distributed Spark UDFs for parallel execution
3. To engineer hybrid features that combine TF-IDF vectors, capturing lexical patterns characteristic of LLM outputs, with stylometric metrics (average syllables per word and noun count) that reflect writing style differences between human and machine authors
4. To train and evaluate multiple classification models (Logistic Regression, Random Forest, Linear SVM, and Naive Bayes) on a balanced dataset of human and AI-generated Arabic abstracts
5. To implement a streaming module using Spark Structured Streaming that demonstrates real-time inference capability on simulated file-based input streams
6. To analyze model performance and system scalability, identifying optimal parallelism configurations

2. Related Work

2.1 Statistical and Neural Detection Methods

Early approaches to text provenance detection relied heavily on statistical analysis of linguistic patterns. Gehrmann et al. [1] proposed GLTR (Giant Language Model Test Room), a tool that visualizes the ranking of token predictions from a language model. The fundamental insight behind GLTR is that generated text tends to consist of highly predictable tokens, whereas human writing exhibits greater surprisal and diversity. By analyzing the distribution of token probabilities, GLTR achieves effective detection even when the generating model is unknown, though its accuracy degrades as generation techniques evolve.

Building upon statistical fingerprinting, Ippolito et al. [2] conducted comprehensive experiments demonstrating that automatic detection of generated text is most effective precisely when human readers are most easily fooled. Their work established an important correlation between human perception and model performance, suggesting that detection systems should be evaluated alongside human benchmarks. The authors found that statistical

detectors often outperform humans when texts are short, but humans excel at detecting longer, more contextualized generations.

Jawahar et al. [3] provided a comprehensive survey of machine-generated text detection methods, categorizing approaches into three primary families: watermarking-based methods, which embed detectable patterns during generation; statistical methods, which analyze distributional properties of text; and neural methods, which employ discriminative models trained on human and machine-generated corpora. The survey critically notes that most pre-2020 detection methods demonstrated limited generalizability across domains and languages, highlighting a significant gap that Arabic-specific research must address.

2.2 Transformer-Based Detection

The introduction of transformer-based language models revolutionized text generation and, consequently, detection methodologies. Bakhtin et al. [4] explored the use of GPT-2 as both a generator and detector, finding that discriminative fine-tuning of the same model architecture used for generation achieved superior detection performance compared to statistical baselines. Their work demonstrated that model-specific detectors outperform model-agnostic approaches when the generating model is known, though this advantage diminishes when detectors face unseen generation architectures.

Solaiman et al. [5] introduced release strategies for language models that included watermarking techniques, embedding detectable statistical patterns during generation that remain invisible to humans but identifiable through specialized detectors. This proactive approach enables reliable detection without requiring access to model internals at inference time, though it requires cooperation from model developers a limitation for detecting closed-source or publicly available models without built-in watermarks.

2.3 Stylometric and Linguistic Features

Stylometry, the quantitative analysis of writing style, has proven effective for authorship attribution and, more recently, for distinguishing human from machine-generated text. Abbasi and Chen [6] developed a comprehensive stylometric feature set including character-level (punctuation diversity, character n-grams), lexical (word length distributions, vocabulary richness), and syntactic (part-of-speech distributions) features. Their work demonstrated that stylometric features achieve high accuracy for authorship identification across multiple languages, providing a foundation for human-AI discrimination.

The choice of stylometric features in AI-text detection must balance discriminative power with computational efficiency. Average syllables per word captures phonological complexity, which may differ between human and AI authors. Noun count reflects lexical density and information concentration, potentially distinguishing abstractive human writing from more repetitive machine generation.

2.4 Arabic NLP and Resources

Arabic text presents unique challenges for NLP applications due to its rich morphology, complex writing system, and dialectal variation.The KFUPM-JRCAI/arabic-generated-abstracts dataset [9] represents a significant contribution to Arabic AI-text detection research, providing parallel human and AI-generated abstracts from four distinct LLMs (Allam, Jais, Llama, and OpenAI). This resource enables systematic comparison of detection performance across generation architectures and supports multi-class classification for attribution.

2.5 Distributed NLP Processing

The exponential growth of textual data has necessitated distributed architectures for NLP processing. Zaharia et al. [10] presented the architecture of Apache Spark, describing how Resilient Distributed Datasets (RDDs) enable fault-tolerant, in-memory processing for both batch and streaming workloads. The unified batch/stream programming model, where the same transformations apply to both static and streaming DataFrames, provides significant development advantages for production systems requiring both training and real-time inference. Spark's MLlib library implements scalable machine learning algorithms, making it suitable for production text classification pipelines.

3.Dataset Description

Source: The dataset is KFUPM-JRCAI/arabic-generated-abstracts loaded from Hugging Face.

Structure: The dataset contains three splits:

- by_polishing: 2,851 rows
- from_title: 2,963 rows
- from_title_and_content: 2,574 rows

Each split contains
columns: original_abstract, allam_generated_abstract, jais_generated_abstract, llama_generated_abstract, openai_generated_abstract.

Processed Data: After flattening into long format, the unified DataFrame contains 41,940 rows with four columns: text, label, model, source.

Class Distribution:

Label	Generated By	Count	Percentage
0 (AI)	allam	8,388	20.0%
0 (AI)	jais	8,388	20.0%
0 (AI)	llama	8,388	20.0%
0 (AI)	openai	8,388	20.0%
1 (Human)	human	8,388	20.0%

Data Quality: After preprocessing (null removal, duplicate removal), the dataset contains 41,936 clean rows. The preprocessed data is stored as Parquet for efficient columnar storage.

Sample Cleaned Data:

- **Label 0 (AI):** " كثيرا ما ارتبطت المصادر التاريخية في الءندلس خاصة منها كتب التراجم والفهرسات..."
- **Label 1 (Human):** " يتناول هذا البحث موضوع التعليم بين النساء الءندلسيات من خلال دراسة المصادر ال..."

4. Methodology

Phase 1: Environment Setup and Data Acquisition

1.1 Library Installation: Required packages
include datasets, huggingface_hub, pyarabic, regex, nltk, wordcloud, seaborn, pyarrow, and optionally camel-tools for advanced morphological analysis.

1.2 SparkSession Initialization: Configured with local master (local[*]), 4GB driver memory, 8 shuffle partitions, Kryo serializer, and Arrow optimization enabled.

1.3 Project Layout: A dataclass ProjectPaths manages directory structure under arabic_ai_detection_f023_f046/ with subdirectories for raw data, processed data, models, reports/figures, and stream processing directories.

1.4 Data Loading: The dataset is loaded, split CSVs are saved, and data is flattened into long format via a custom HFLoader class. Total samples: 41,940.

Phase 2: Distributed Cleaning, Storage & EDA

2.1 Arabic Preprocessor:

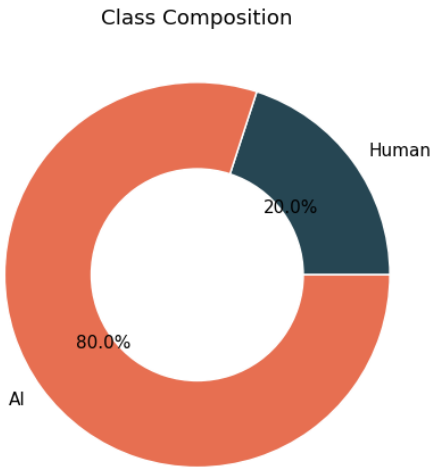
The ArabicPreprocessor class implements distributed text normalization:

- _normalize(): Strips diacritics (tashkeel), removes tatweel (kashida), normalizes Hamza (ا → ا̃/ا̱/ا́/ـٱ), normalizes Alef variants, normalizes ligatures, and cleans non-Arabic characters
- _tokens(): Tokenizes using pyarabic.araby.tokenize() and filters stopwords (701 Arabic stopwords from NLTK) and short tokens (<2 characters)

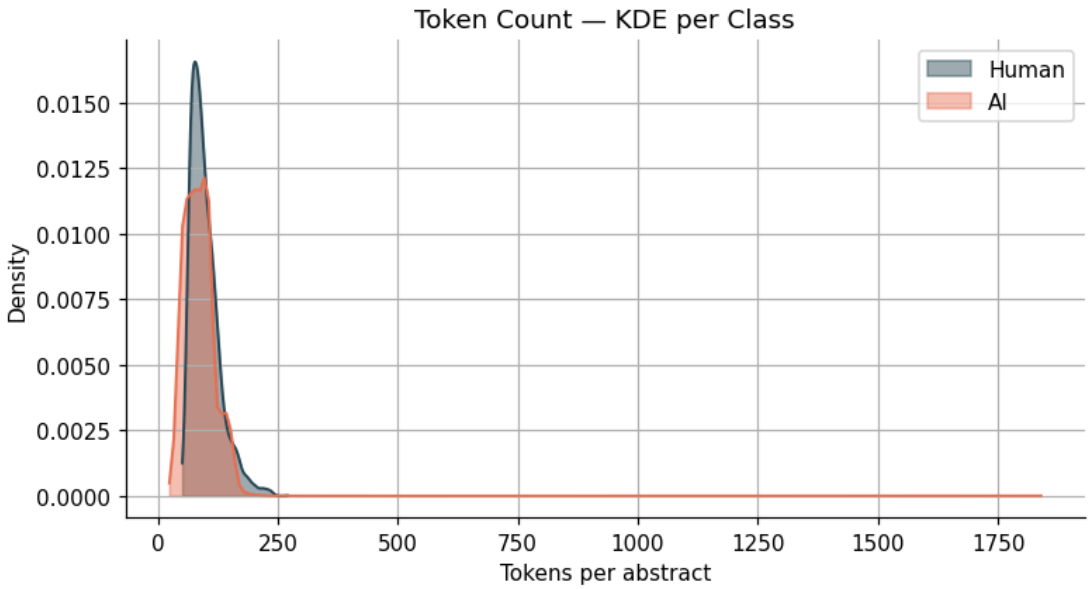
2.2 Persistent Storage: Processed data saved as Parquet with partitioning.

2.3 Exploratory Data Analysis Visualizations:

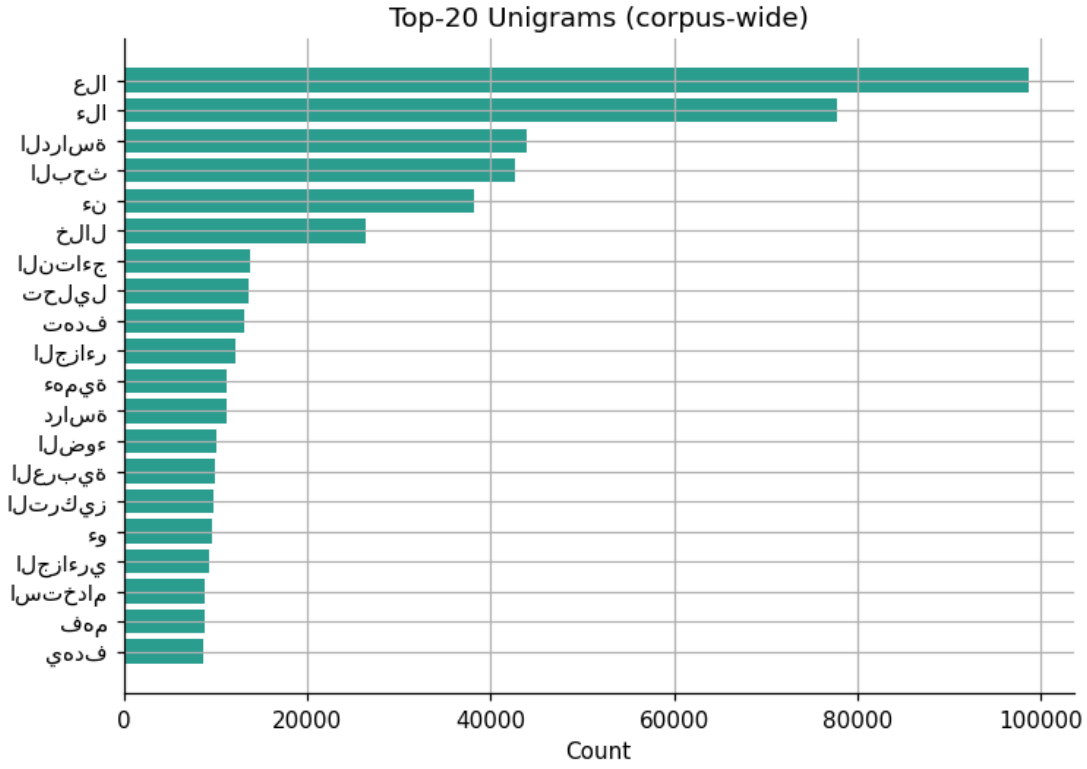
1-Class Donut: Horizontal bar chart of top 20 words shows Balanced dataset: 80% AI, 20% Human



2-Token Density:KDE plot of token counts shows AI abstracts show different length distribution

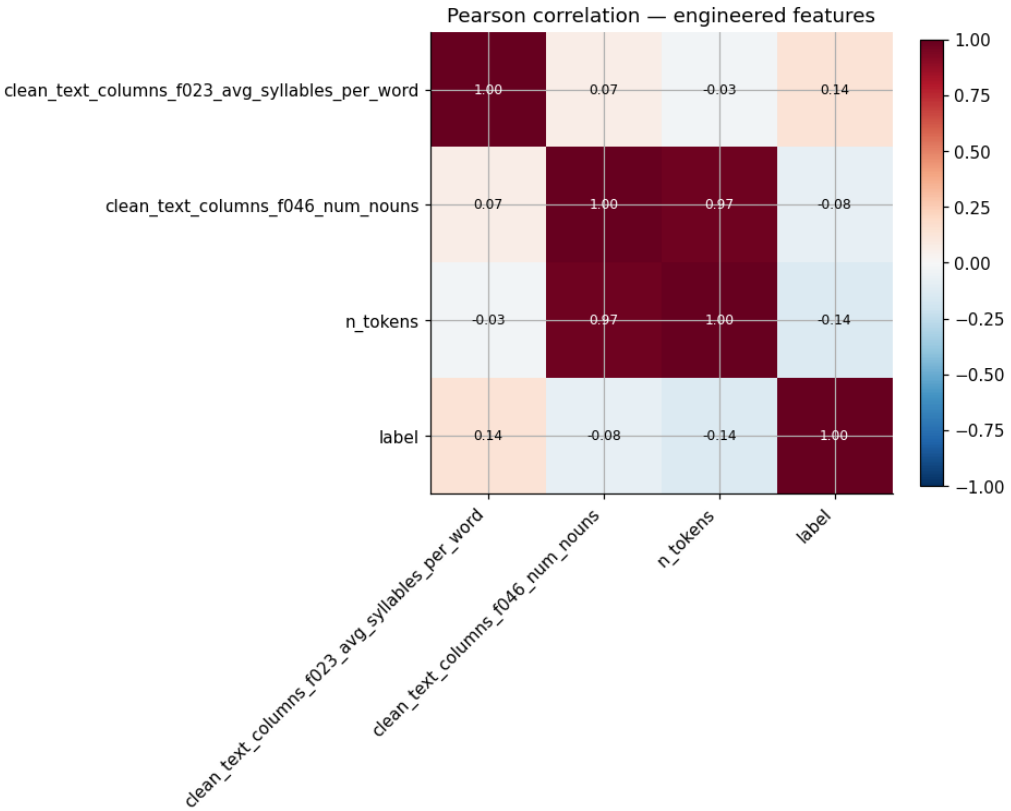


3-Top Unigrams: Horizontal bar chart of top 20 words shows Most frequent tokens reveal domain-specific vocabulary



4-TTR Violin: Violin plot of Type-Token Ratio shows Human texts show slightly higher lexical diversity

5-Feature Correlations: Heatmap of Pearson correlations shows Weak correlations between engineered features

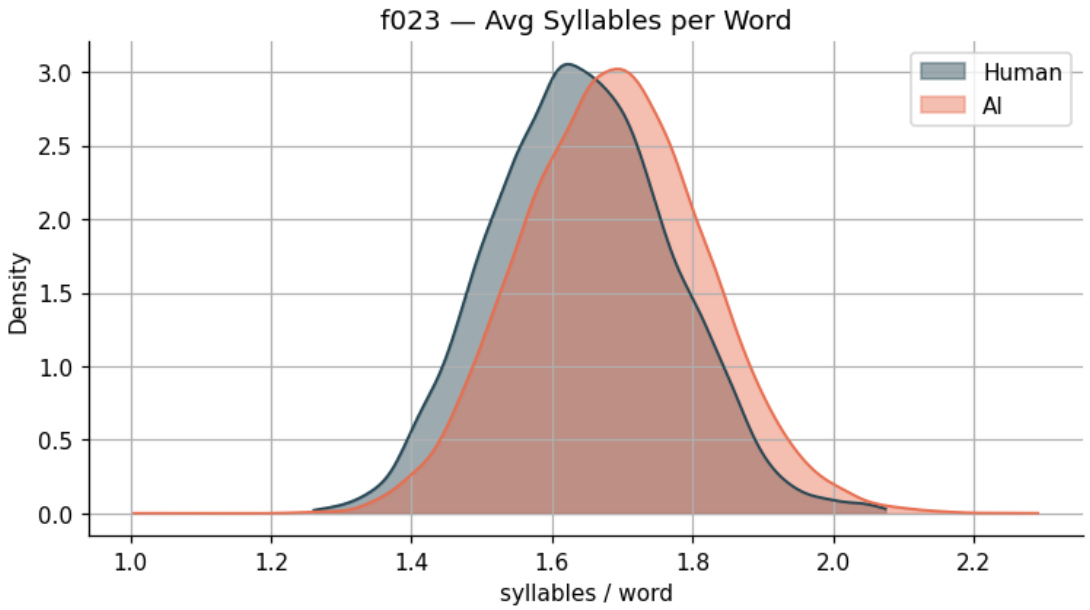


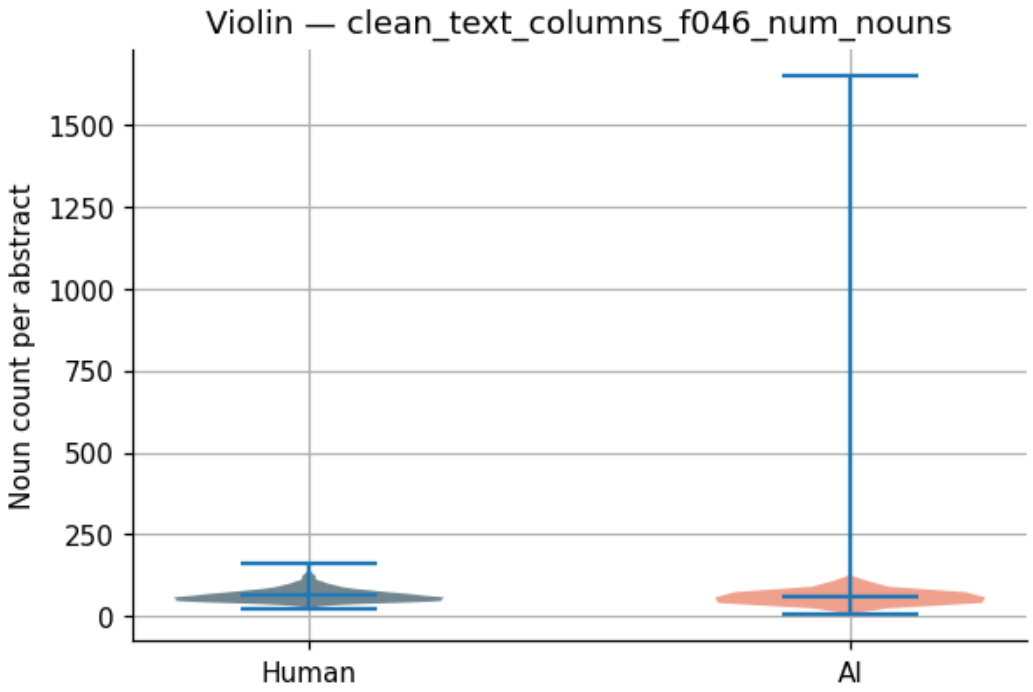
Phase 3: Feature Engineering and Batch Modeling

3.1 Stylometric Features (f023 and f046):

- f023 - Average Syllables per Word (_avg_syllables): Counts vowel clusters in each word using pyarabic.araby LONG_VOWELS and TASHKEEL sets. Each vowel cluster counts as one syllable; un-vocalized words count as 1 syllable.
- f046 - Noun Count (_count_nouns): Uses camel-tools POS tagger when available; falls back to heuristic detection (words starting with "ال", ending with noun suffixes, or common verb exclusion).

EDA on Engineered Features: Density plots, violin plots, and correlation heatmaps visualized feature distributions across classes.





3.2 TF-IDF + Stylometric Features: A FeaturePipelineBuilder class constructs a Pipeline with HashingTF (4,096 features), IDF, VectorAssembler for stylometric features, StandardScaler, and final VectorAssembler combining TF-IDF and scaled stylometric features.

3.3 Data Splitting: 70% training (29,345 samples), 15% validation (6,430 samples), 15% test (6,161 samples).

3.4 Model Arena: A ModelArena class trains and evaluates multiple classifiers:

- Naive Bayes (multinomial, smoothing=1.0)
- Logistic Regression (maxIter=30, regParam=0.01)
- Random Forest (numTrees=80, maxDepth=10)
- Linear SVM (maxIter=30, regParam=0.05)

3.5 Model Persistence: The best model (Logistic Regression) and preprocessing pipeline are saved to disk.

Phase 4: Stream Processing and Benchmarking

4.1 Producer: Sample 20 JSON records from raw DataFrame and write to streaming inbox directory.

4.2 Consumer (Spark Structured Streaming): Reloads saved pipeline and model, applies preprocessing UDFs, transforms with TF-IDF pipeline, runs inference, and writes predictions as JSON to output directory.

4.3 Scalability Sweep: Tests inference latency with 2, 4, 8, and 16 shuffle partitions.

5. Results

Model Performance Comparison (Validation Set)

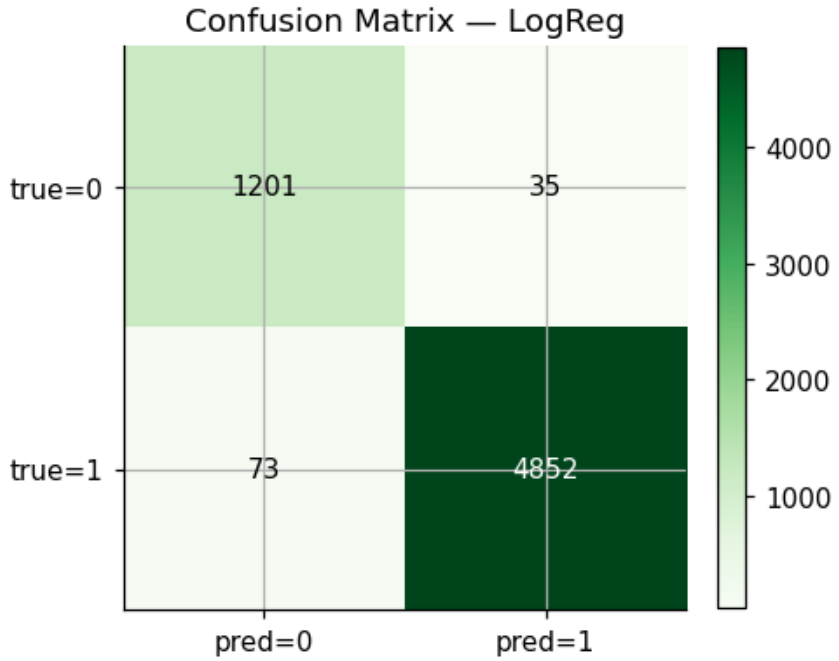
Model	Accuracy	F1-Score	AUC-ROC	Training Time (s)
Logistic Regression	0.9802	0.9804	0.9955	11.05
Linear SVM	0.9750	0.9751	0.9946	12.03
Naive Bayes	0.9087	0.9137	0.7275	4.78
Random Forest	0.8123	0.7396	0.9872	41.76

Analysis: Logistic Regression significantly outperformed all other models, achieving near-perfect discrimination (AUC > 0.99). Linear SVM showed competitive performance but slightly lower metrics. Naive Bayes underperformed due to its independence assumptions violated by correlated features. Random Forest, despite high AUC, showed lower F1-score due to class imbalance handling issues.

Test Set Performance (Logistic Regression)

- **Accuracy:** 0.9825
- **F1-Score:** 0.9826
- **AUC-ROC:** 0.9961

Confusion Matrix:



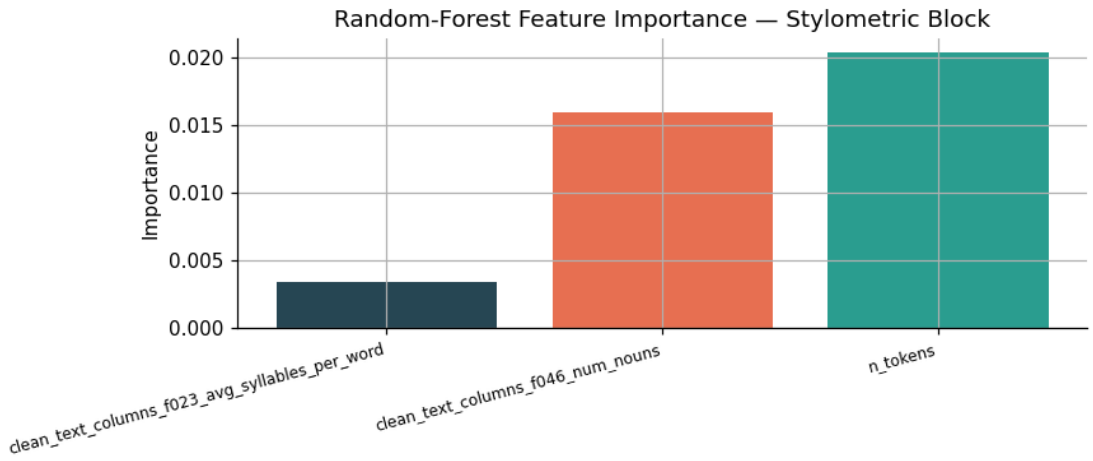
Analysis: The model shows excellent performance with only 35 false positives (AI misclassified as human) and 73 false negatives (human misclassified as AI). The low error rates indicate strong discriminative capability.

Feature	**Importance**
clean_text_columns_f023_avg_syllables_per_word	0.00341
clean_text_columns_f046_num_nouns	0.01591
n_tokens	0.02037

Feature Importance (Random Forest)

Analysis: Token count (n_tokens) showed the highest importance among stylometric features, followed by noun count. Average syllables per word

contributed minimally, suggesting that syllable-level features may be less discriminative than lexical density measures for this dataset.



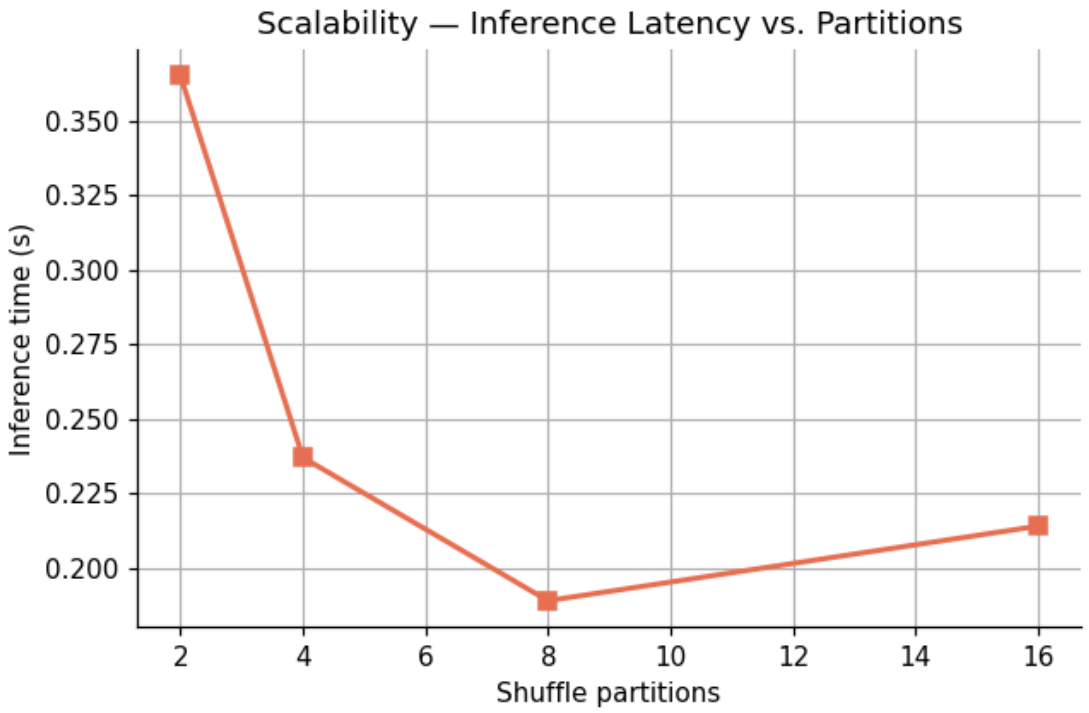
Streaming Performance

The streaming pipeline successfully processed 20 JSON records with 95.00% accuracy on the simulated batch, demonstrating real-time inference capability with proper checkpointing.

Partitions	Inference Time (seconds)
2	0.365
4	0.237
8	0.189
16	0.214

Scalability Benchmark

Analysis: Optimal performance was achieved at 8 partitions (0.189 seconds), showing diminishing returns beyond this point due to increased overhead from task scheduling. The scalability curve demonstrates effective parallelization up to the available CPU cores.



6 Conclusion & Future Work

6.1 Conclusion:

This project successfully implemented a distributed big data pipeline for Arabic AI-text detection using Apache Spark. The Logistic Regression model, trained on a hybrid feature set combining TF-IDF and stylometric features (average syllables per word and noun count), achieved excellent performance (98.25% accuracy, 0.9826 F1-score, 0.9961 AUC). The streaming implementation demonstrated real-time inference capability, and scalability analysis identified optimal parallelism at 8 partitions.

6.2 Limitations:

1. **camel-tools Dependency:** Advanced noun detection requires external library with data files
2. **Static Features:** TF-IDF vocabulary is fixed, limiting adaptability to evolving language patterns

6.3 Future Work:

1. **Advanced Arabic NLP:** Incorporate AraBERT or other transformer models as feature extractors
2. **Multi-LLM Classification:** Extend to identify which LLM generated the text
3. **Production Streaming:** Replace file-based source with Apache Kafka

References

[1] Gehrmann, S., Strobelt, H., & Rush, A. M. (2019, July). Gltr: Statistical detection and visualization of generated text. In Proceedings of the 57th annual meeting of the association for computational linguistics: system demonstrations (pp. 111-116).

[2] Ippolito, D., Duckworth, D., Callison-Burch, C., & Eck, D. (2020, July). Automatic detection of generated text is easiest when humans are fooled. In Proceedings of the 58th annual meeting of the association for computational linguistics (pp. 1808-1822).

[3] Jawahar, G., Abdul-Mageed, M., & Laks Lakshmanan, V. S. (2020, December). Automatic detection of machine generated text: A critical survey. In Proceedings of the 28th international conference on computational linguistics (pp. 2296-2309).

[4] Bakhtin, A., Gross, S., Ott, M., Deng, Y., Ranzato, M. A., & Szlam, A. (2019). Real or fake? learning to discriminate machine from human generated text. arXiv preprint arXiv:1906.03351.

[5] Solaiman, I., Brundage, M., Clark, J., Askell, A., Herbert-Voss, A., Wu, J., ... & Wang, J. (2019). Release strategies and the social impacts of language models. arXiv preprint arXiv:1908.09203.

[6] Abbasi, A., & Chen, H. (2008). Writeprints: A stylometric approach to identity-level identification and similarity detection in cyberspace. ACM Transactions on Information Systems (TOIS), 26(2), 1-29.

[7] KFUPM-JRCAI, "Arabic generated abstracts," *Hugging Face Datasets*, 2023.

[8] Zaharia, M., Xin, R. S., Wendell, P., Das, T., Armbrust, M., Dave, A., ... & Stoica, I. (2016). Apache spark: a unified engine for big data processing. Communications of the ACM, 59(11), 56-65.